Healthcare Data Analysis Project

1. Project Overview

This project analyzes healthcare patient records to uncover insights related to patient demographics, disease prevalence, hospital operations, treatment costs, insurance coverage, and quality of care. The objective is to support data-driven decision-making for hospitals and healthcare providers by identifying patterns in patient admissions, medical conditions, billing trends, and operational efficiency.

---

2. Dataset Summary

- Rows: 55,500
- Columns: 16

Key Features

- Patient Information (Patient Name, Age, Gender, Blood Type)
- Medical Information (Medical Condition, Medication, Test Results)
- Hospital Information (Hospital, Doctor, Room Number)
- Financial Information (Billing Amount, Insurance Provider)
- Admission Information (Admission Type, Date of Admission, Discharge Date)
- Operational Metrics (Length of Stay)

Data Quality Checks

- Checked for missing values.
- Checked for duplicate records.
- Standardized column names and formats.
- Converted date columns into proper datetime format.

---

3. Exploratory Data Analysis using Python

The dataset was cleaned, transformed, and prepared using Python before loading it into MySQL for SQL analysis.

Data Loading

- Imported required libraries such as Pandas, NumPy, Matplotlib, and SQLAlchemy.
- Loaded the healthcare dataset into a Pandas DataFrame.
- Create engine to established connection between Jupyter Notebook and Mysql

[36]:
```
import mysql.connector
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sqlalchemy import create_engine
```

[2]:
```
# create connection between jupyter notebook and mysql
```

[37]:
```
engine = create_engine("mysql+pymysql://root:Monika11751#@localhost/healthcare")
```

Initial Exploration

- Used `df.info()` to understand data structure.
- Used `df.head()` and `df.describe()` to examine sample records and summary statistics.

[38]:
```
df = pd.read_csv("Healthcare_dataset.csv")
```

[39]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 55500 entries, 0 to 55499
Data columns (total 16 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Name                55500 non-null  object
 1   Age                 55500 non-null  int64
 2   Gender              55500 non-null  object
 3   Blood Type          55500 non-null  object
 4   Medical Condition   55500 non-null  object
 5   Date of Admission   55500 non-null  object
 6   Doctor              55500 non-null  object
 7   Hospital            55500 non-null  object
 8   Insurance Provider  55500 non-null  object
 9   Billing Amount      55500 non-null  float64
 10  Room Number         55500 non-null  int64
 11  Admission Type      55500 non-null  object
 12  Discharge Date      55500 non-null  object
 13  Medication          55500 non-null  object
 14  Test Results        55500 non-null  object
 15  Length of Stay      55500 non-null  int64
dtypes: float64(1), int64(3), object(12)
memory usage: 6.8+ MB
```

[40]:
```
df.head()
```

[40]:

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	I
0	Bobby Jackson	19	Female	AB+	Infections	1/31/2024	Matthew Smith	Northwestern Memorial Hospital	Blue Cross	2212.272701	328	Emergency	2/7/2024	Azithromycin	I
1	Leslie Terry	15	Female	B-	Flu	8/20/2019	Samantha Davies	UI Health (University of Illinois Hospital)	UnitedHealthcare	3185.161388	265	Emergency	8/22/2019	Tamiflu	Ab
2	Danny Smith	50	Female	A+	Cancer	9/22/2022	Tiffany Mitchell	UI Health (University of Illinois Hospital)	Blue Cross	72055.214060	205	Elective	10/30/2022	Cisplatin	Incon
3	Andrew Watts	24	Female	O+	Asthma	11/18/2020	Kevin Wells	UI Health (University of Illinois Hospital)	Aetna	4092.601229	450	Elective	11/19/2020	Prednisone	I
4	Adrienne Bell	80	Female	A+	Heart Disease	9/19/2022	Kathleen Hanna	Northwestern Memorial Hospital	Cigna	47985.660250	458	Routine	10/27/2022	Beta-blockers	Incon

```
[41]: df.describe()
```

[41]:

	Age	Billing Amount	Room Number	Length of Stay
count	55500.000000	55500.000000	55500.000000	55500.000000
mean	48.001622	21835.044636	301.134829	17.841009
std	21.105827	23574.413594	115.243069	20.272294
min	5.000000	500.220989	101.000000	1.000000
25%	32.000000	4195.525741	202.000000	4.000000
50%	50.000000	11089.537760	302.000000	8.000000
75%	65.000000	35887.703407	401.000000	28.000000
max	90.000000	99997.797980	500.000000	89.000000

Data Cleaning

- Checked Null values and duplicate values
- Removed extra spaces from column names.
- Renamed columns for consistency.
- Converted Admission Date and Discharge Date into standard datetime format.

```
[42]: df.isnull().sum()
```

[42]:

```
Name                  0
Age                   0
Gender                0
Blood Type            0
Medical Condition     0
Date of Admission     0
Doctor                0
Hospital              0
Insurance Provider    0
Billing Amount        0
Room Number           0
Admission Type        0
Discharge Date        0
Medication            0
Test Results          0
Length of Stay        0
dtype: int64
```

```
[43]: df.duplicated().sum()
```

[43]:

```
np.int64(0)
```

```
[13]: # Strip extra spaces in column names
```

```
[46]: df.columns = df.columns.str.strip()
```

```
[15]: # Change the column name and relplace ' ' with '_'
```

```
[47]: df.rename(columns={'Name': 'Patient Name'}, inplace=True)
```

```
[48]: df.columns = df.columns.str.replace(' ', '_')
```

```
[19]: # Change the formate of Admission Date and Discharge Date in YYYY-MM-DD

[50]: df['Date_of_Admission'] = pd.to_datetime(df['Date_of_Admission']).dt.strftime('%Y-%m-%d')

[51]: df['Discharge_Date'] = pd.to_datetime(df['Discharge_Date']).dt.strftime('%Y-%m-%d')

[22]: # Change the data type from object to date for both the column

[52]: date_cols = ['Date_of_Admission', 'Discharge_Date']

      for col in date_cols:
          df[col] = pd.to_datetime(df[col])

[53]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 55500 entries, 0 to 55499
Data columns (total 16 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Patient_Name        55500 non-null  object
 1   Age                 55500 non-null  int64
 2   Gender              55500 non-null  object
 3   Blood_Type          55500 non-null  object
 4   Medical_Condition   55500 non-null  object
 5   Date_of_Admission   55500 non-null  datetime64[ns]
 6   Doctor              55500 non-null  object
 7   Hospital            55500 non-null  object
 8   Insurance_Provider  55500 non-null  object
 9   Billing_Amount      55500 non-null  float64
 10  Room_Number         55500 non-null  int64
 11  Admission_Type      55500 non-null  object
 12  Discharge_Date      55500 non-null  datetime64[ns]
```

Data Transformation

- Standardized medical condition values (e.g., Alzheimer's naming consistency).
- Created Age Group categories.
- Created Risk Category classification for patients.

```
[26]: # change Alzheimer's into Alzheimer in medical_condition column

[55]: df['Medical_Condition'] = df['Medical_Condition'].replace(
          "Alzheimer's",
          "Alzheimer"
      )
```

```
[29]: # create new column for diseases risk category

[58]: df['risk_category'] = df['Medical_Condition'].apply(
          lambda x:
          'Critical' if x in ['Cancer', 'Heart Disease', 'Alzheimer']
          else 'High Risk' if x in ['Diabetes', 'Obesity', 'Asthma']
          else 'Low Risk'
      )

[59]: df['age_group'] = df['Age'].apply(
          lambda x:
          'Child' if x <= 18
          else 'Young Adult' if x <= 35
          else 'Adult' if x <= 50
          else 'Senior Adult' if x <= 65
          else 'Elderly'
      )
```

Data Validation

- Verified data types.
- Checked for duplicates.

- Ensured consistency across categorical values

Database Integration

- Established connection between Jupyter Notebook and MySQL.
- Loaded the cleaned healthcare dataset into the MySQL database.

```
[37]: engine = create_engine("mysql+pymysql://root:Monika11751#@localhost/healthcare")

[34]: # Now we will upload data in SQL

[61]: df.to_sql("healthcare_data", engine, if_exists='replace', index=False)

[61]: 55500
```

---

4. Data Analysis using SQL

Structured SQL analysis was performed to answer key healthcare and operational questions.

1. Medical Conditions Generating Highest Treatment Costs

Analyzed total billing amounts associated with each medical condition to identify the most expensive diseases.

```
select medical_condition , round(sum(billing_amount),2) as Total_cost from healthcare_data
group by medical_condition
ORDER BY total_cost desc;
```

Result Grid | Filter Rows:

medical_condition	Total_cost
Cancer	447887395.35
Heart Disease	309902646.64
Alzheimer	223281243.66
Diabetes	87584835.23
Obesity	70328608
Asthma	34715105.99
Flu	19335295.35
Infections	18809847.06

---

2. Most Common Medical Conditions

Identified diseases with the highest number of patients.

```sql
select medical_condition , count(*) as Patient_count from healthcare_data
group by Medical_Condition
ORDER BY count(*) desc;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

medical_condition	Patient_count
Flu	7046
Diabetes	7005
Obesity	6994
Cancer	6940
Asthma	6908
Heart Disease	6900
Alzheimer	6861
Infections	6846

3. Critical Disease Cases by Age Group

Analyzed which age groups are most affected by critical medical conditions.

```sql
select age_group , count(*)as Critical_cases from healthcare_data where risk_category = 'critical'
group by age_group
ORDER BY count(*) desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

age_group	Critical_cases
Elderly	12148
Senior Adult	8118
Adult	435

4. Gender-Based Disease Trends

Examined disease occurrence patterns across male and female patients.

```sql
select Medical_Condition, gender, count(*) as Patient_Count from healthcare_data
WHERE risk_category = 'critical'
group by gender, Medical_Condition
ORDER BY Medical_Condition, count(*) desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Medical_Condition	gender	Patient_Count
Alzheimer	Female	3479
Alzheimer	Male	3382
Cancer	Female	3506
Cancer	Male	3434
Heart Disease	Female	3505
Heart Disease	Male	3395

---

5. Hospitals with Highest Average Billing Amount

Compared average treatment costs across hospitals.

```sql
select hospital , round(avg(billing_amount),2) as avg_billing from healthcare_data
GROUP BY Hospital
ORDER BY round(avg(billing_amount),2) desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

hospital	avg_billing
Northwestern Memorial Hospital	21939.53
Loyola University Medical Center	21898.35
UI Health (University of Illinois Hospital)	21780.47
UChicago Medicine	21721.82

---

6. Hospitals Generating Highest Revenue

Identified hospitals contributing the highest overall healthcare revenue.

```sql
select hospital , round(sum(billing_amount),2) as Total_Revenue from healthcare_data
GROUP BY Hospital
ORDER BY Total_Revenue desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

hospital	Total_Revenue
Northwestern Memorial Hospital	308689213.32
UI Health (University of Illinois Hospital)	303837582.38
UChicago Medicine	301281681.1
Loyola University Medical Center	298036500.47

---

7. Insurance Providers Covering Costliest Treatments

Analyzed insurance provider expenditure across medical conditions.

```sql
select insurance_provider , medical_condition , round(sum(billing_amount),2) as Total_expense
from healthcare_data
group by insurance_provider , medical_condition
ORDER BY Total_expense desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

insurance_provider	medical_condition	Total_expense
Cigna	Cancer	90984306.24
Blue Cross	Cancer	90687019.83
UnitedHealthcare	Cancer	89812904.11
Medicare	Cancer	88336350.49
Aetna	Cancer	88066814.68

---

8. Admission Types with Longest Hospital Stay

Compared average patient stay durations across admission categories.

```sql
SELECT
    admission_type,
    ROUND(AVG(length_of_stay),2) AS avg_stay_days
FROM healthcare_data
GROUP BY admission_type
ORDER BY avg_stay_days DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

admission_type	avg_stay_days
Routine	20.75
Elective	18.50
Urgent	17.97
Emergency	15.84

---

9. Hospitals with Highest Emergency Admissions

Identified hospitals handling the largest number of emergency cases.

```sql
SELECT
    hospital,
    COUNT(*) AS emergency_cases
FROM healthcare_data
WHERE admission_type = 'Emergency'
GROUP BY hospital
ORDER BY emergency_cases DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

hospital	emergency_cases
Northwestern Memorial Hospital	5122
UChicago Medicine	5042
UI Health (University of Illinois Hospital)	5004
Loyola University Medical Center	4936

---

10. Factors Influencing Length of Stay

Investigated the relationship between medical conditions, risk categories, admission types, and hospital stay duration.

```sql
SELECT
    medical_condition,
    admission_type,
   risk_category,
    ROUND(AVG(length_of_stay),2) AS avg_length_of_stay
FROM healthcare_data
GROUP BY medical_condition, admission_type, risk_category
ORDER BY avg_length_of_stay DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

medical_condition	admission_type	risk_category	avg_length_of_stay
Alzheimer	Urgent	Critical	54.96
Alzheimer	Elective	Critical	54.93
Alzheimer	Emergency	Critical	54.17
Alzheimer	Routine	Critical	53.98
Cancer	Routine	Critical	37.19
Cancer	Emergency	Critical	36.64
Cancer	Urgent	Critical	36.64
Cancer	Elective	Critical	36.13

11. Medication Effectiveness Analysis

Compared medications against patient test results.

```sql
SELECT
    medication,
    test_results,
    COUNT(*) AS patient_count
FROM healthcare_data
GROUP BY medication, test_results
ORDER BY medication, patient_count DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

medication	test_results	patient_count
Albuterol	Abnormal	1143
Albuterol	Normal	1119
Amoxicillin	Abnormal	1159
Amoxicillin	Normal	1152
Aspirin	Inconclusive	1178
Aspirin	Abnormal	1159
Azithromycin	Normal	1165

12. Conditions with Highest Abnormal Test Results

Identified diseases frequently associated with abnormal test outcomes.

```
98  SELECT
99      medical_condition,
100     COUNT(*) AS abnormal_cases
101 FROM healthcare_data
102 WHERE test_results = 'Abnormal'
103 GROUP BY medical_condition
104 ORDER BY abnormal_cases DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

medical_condition	abnormal_cases
Obesity	3524
Flu	3515
Cancer	3502
Heart Disease	3475
Asthma	3466
Alzheimer	3461
Diabetes	3443
Infections	3398

---

13. Hospitals with Highest Inconclusive Test Results

Examined healthcare facilities showing a high number of inconclusive tests.

```
108 SELECT
109     hospital,
110     COUNT(*) AS inconclusive_cases
111 FROM healthcare_data
112 WHERE test_results = 'Inconclusive'
113 GROUP BY hospital
114 ORDER BY inconclusive_cases DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

hospital	inconclusive_cases
UChicago Medicine	2598
Northwestern Memorial Hospital	2597
UI Health (University of Illinois Hospital)	2574
Loyola University Medical Center	2494

---

5. Power BI Dashboard

An interactive Power BI dashboard was developed to visualize healthcare insights and KPIs.

Dashboard KPIs

- Average Billing Amount
- Average Length of Stay
- Emergency Admission Rate
- Patient Count
- Total Revenue

```
Average_Billing = CALCULATE(AVERAGE(healthcare_data[Billing_Amount]))
```

```
Average_length_of_stay = CALCULATE(AVERAGE(healthcare_data[Length_of_Stay]))
```

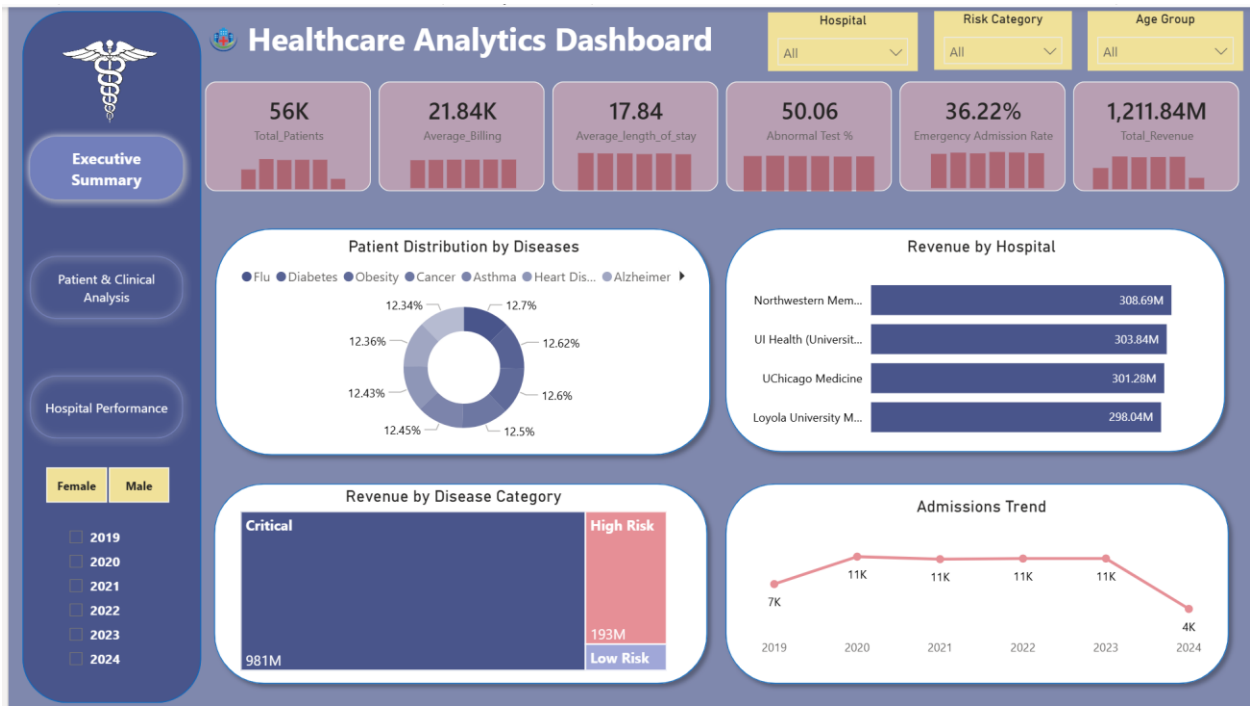
```
Emergency Admission Rate =
DIVIDE(
    CALCULATE(
        COUNTROWS(healthcare_data),
        healthcare_data[admission_type] = "Emergency"
    ),
    COUNTROWS(healthcare_data)
)
```

```
Total_Patients = CALCULATE(COUNTROWS(healthcare_data))
```

```
Total_Revenue = CALCULATE(SUM(healthcare_data[Billing_Amount]))
```

Dashboard Visualizations

- Revenue by Hospital
- Patients by Medical Condition
- Emergency Admission Analysis
- Insurance Provider Analysis
- Age Group Distribution
- Billing Trends
- Hospital Occupancy Trend
- Test Result Distribution


Healthcare Analytics Dashboard
Hospital
All
Risk Category
All
Age Group
All
56K
Total_Patients
21.84K
Average_Billing
17.84
Average_length_of_stay
50.06
Abnormal Test %
36.22%
Emergency Admission Rate
1,211.84M
Total_Revenue
Executive Summary
Patient & Clinical Analysis
Hospital Performance
Female
Male
2019
2020
2021
2022
2023
2024
Patient Distribution by Diseases
Flu
Diabetes
Obesity
Cancer
Asthma
Heart Dis...
Alzheimer
12.34%
12.7%
12.36%
12.62%
12.43%
12.6%
12.45%
12.5%
Revenue by Hospital
Northwestern Mem...
308.69M
UI Health (Universit...
303.84M
UChicago Medicine
301.28M
Loyola University M...
298.04M
Revenue by Disease Category
Critical
981M
High Risk
193M
Low Risk
Admissions Trend
7K
11K
11K
11K
11K
4K
2019
2020
2021
2022
2023
2024


Total Critical Cases
0K
21K
41K
Total Critical Cases
Average_Billing
42K
35K
12K
8K
9K
0K
3K
5K
Elderly
Senior Adult
Adult
Child
Young Adult
age_group
Executive Summary
Patient & Clinical Analysis
Hospital Performance
Female
Male
2019
2020
2021
2022
2023
2024
Age Group Analysis
Senior Adult
13.8K
Elderly
13.3K
Adult
11.8K
Young Adult
10.3K
Child
6.3K
Disease Severity by Age Group
Adult
Child
Elderly
Senior Adult
Young Adult
High Risk
33.21%
9.65%
26.98%
24.71%
Critical
58.68%
39.22%
Low Risk
32.01%
30.81%
37.18%
Medical_Condition Female Male
Alzheimer 3479 3382
Asthma 3422 3486
Cancer 3506 3434
Diabetes 3505 3500
Flu 3531 3515
Heart Disease 3505 3395
Infections 3459 3387
Obesity 3548 3446
Abnormal
Normal
Inconclusive
10K
28K
17K
Critical Disease Analysis
High Risk
21K
Critical
21K
Low Risk
14K

6. Key Findings

- Certain medical conditions contribute significantly higher treatment costs.
- Critical cases are concentrated within specific age groups.
- Emergency admissions account for a substantial portion of hospital activity.
- Billing amounts vary significantly across hospitals.
- Some insurance providers bear a larger share of healthcare expenses.
- Certain conditions are more frequently associated with abnormal or inconclusive test results.

7. Business Recommendations

Improve Preventive Care

Focus on early detection and prevention programs for high-cost medical conditions.

Optimize Hospital Resources

Allocate beds, staff, and emergency resources based on admission patterns and occupancy trends.

Strengthen Insurance Partnerships

Collaborate with insurance providers managing high-cost treatment categories.

Reduce Length of Stay

Develop care pathways for conditions associated with extended hospitalization.

Enhance Diagnostic Accuracy

Investigate hospitals with high rates of inconclusive test results and improve diagnostic processes.

Monitor Critical Patient Segments

Develop targeted healthcare programs for age groups showing elevated critical case rates.

---

8. Conclusion

The Healthcare Data Analysis Project demonstrates how Python, MySQL, and Power BI can be integrated to transform raw healthcare records into actionable insights. The analysis supports strategic planning, operational optimization, cost management, and improved patient care outcomes.